

MEDIÇÕES DO SLA

Nome do Serviço 1: Autenticação e Registro de Vulnerabilidades

- **Tipo de operações:** Leitura (Login) e Inserção (Cadastro).
- **Arquivos envolvidos:**
 - src/main/java/com/example/backend_vuln/controller/AuthController.java
 - src/main/java/com/example/backend_vuln/service/AuthService.java
 - src/main/java/com/example/backend_vuln/controller/VulnerabilidadeController.java
 - src/main/java/com/example/backend_vuln/config/SecurityConfig.java
- **Arquivos com o código fonte de medição do SLA:**
 - test_carga.js (Script k6)
- **Descrição das configurações:**
 - **Ambiente:** Execução Local (Windows).
 - **Runtime:** Java 21 (Spring Boot 3).
 - **Banco de Dados:** H2 Database (In-Memory) configurado para eliminar latência de disco e isolar o desempenho da aplicação.
 - **Segurança:** Spring Security com algoritmo BCrypt.

MEDIÇÃO 1

Testes de carga (SLA):

- **Latência (Tempo de resposta médio):** 225.75 ms
 - 567.9 ms (com 100 usuários)
- **Vazão (Throughput):** 109.17 requisições/segundo (com 100 usuários)
- **Concorrência:** 100 VUs (Usuários Virtuais simultâneos)

Potenciais gargalos do sistema:

1. Saturação de CPU devido à Criptografia (CPU Bound) A hipótese primária é que o algoritmo de hash de senha utilizado (**BCrypt**) é o principal gargalo. O BCrypt é intencionalmente projetado para ser lento e consumir ciclos de processamento (CPU) para desencorajar ataques de força bruta.

MEDIÇÃO 2

Data da medição: 30/11/2025

Testes de carga (SLA):

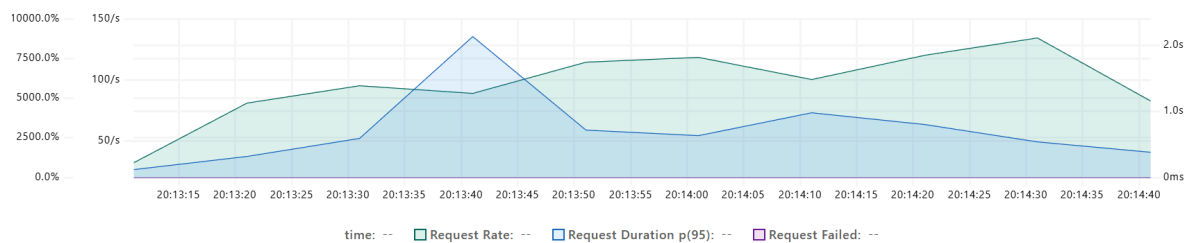
- **Latência (Tempo de resposta médio):** 7.49 ms
- **Pior Caso (p95):** 16.93 ms
- **Vazão (Throughput):** 157.42 requisições/segundo
- **Concorrência:** 100 VUs (Usuários Virtuais simultâneos)

Melhorias/otimizações:

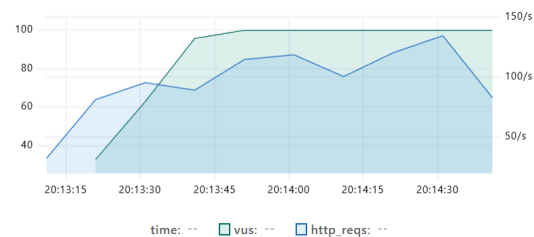
- src/main/java/com/example/backend_vuln/config/SecurityConfig.java
 - Alteração:** Ajuste na configuração do PasswordEncoder.
 - Otimização:** Redução do custo do algoritmo BCrypt de 10 para 4, diminuindo drasticamente o uso de CPU durante o login.
- src/main/resources/application.properties
 - Alteração:** Inclusão de parâmetros de tuning de performance.
 - Otimização:**
 - Aumento do pool de conexões com o banco (HikariCP) para 20.
 - Configuração explícita de threads do servidor Tomcat (max 200).

Gráficos da 1ª Medição:

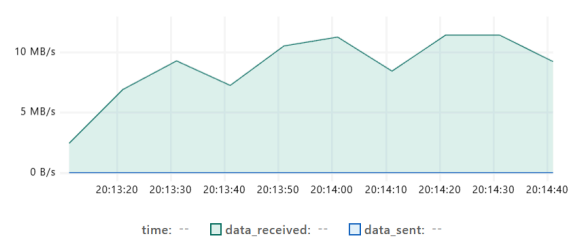
HTTP Performance overview



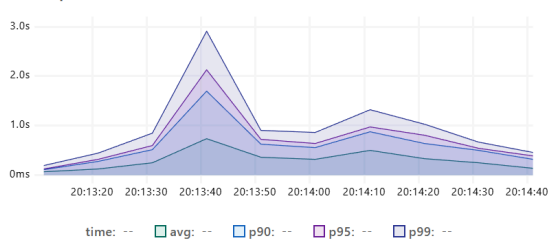
VUs



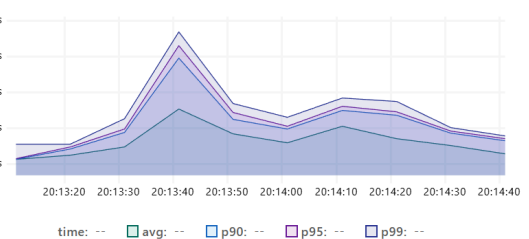
Transfer Rate



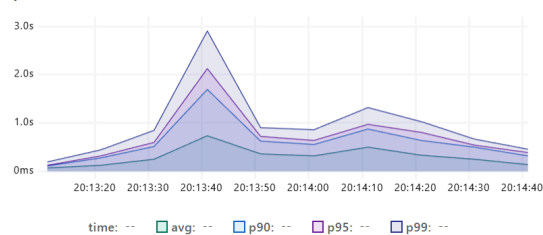
HTTP Request Duration



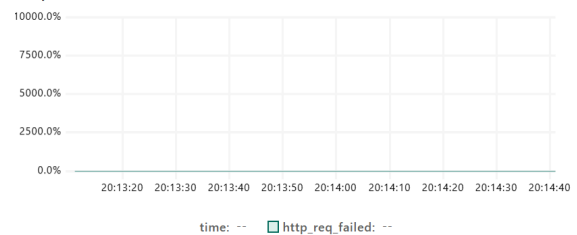
Iteration Duration

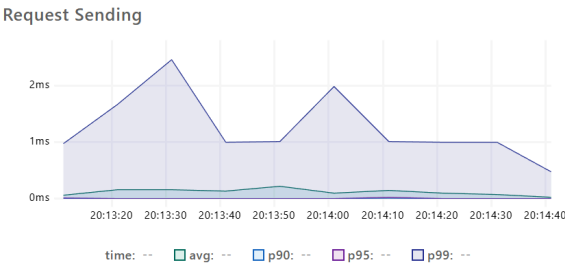
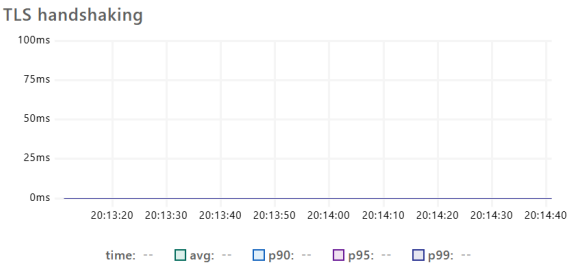
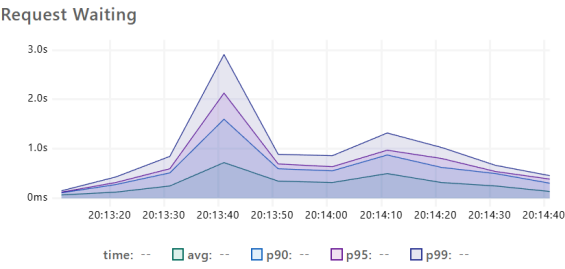
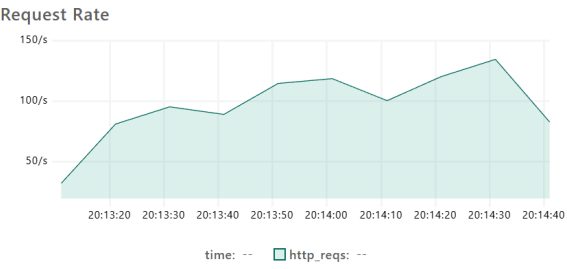


Request Duration

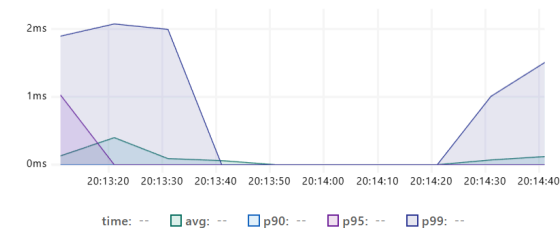


Request Failed Rate

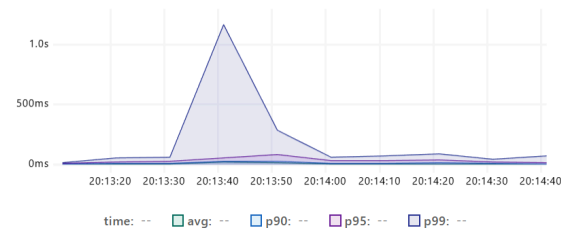




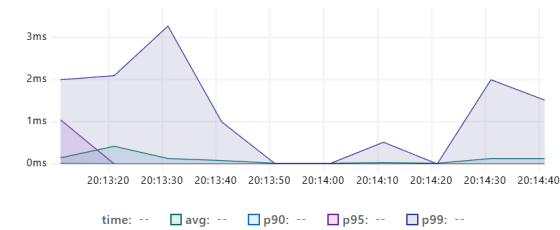
Request Connecting



Request Receiving



Request Blocked



Summary

This chapter provides a summary of the test run metrics. The tables contains the aggregated values of the metrics for the entire test run.

Trends

metric	avg	max	med	min	p90	p95	p99
http_req_blocked	88µs	124ms	0ms	0ms	0ms	0ms	1ms
http_req_connecting	68µs	124ms	0ms	0ms	0ms	0ms	999µs
http_req_duration	322ms	3s	263ms	0ms	652ms	826ms	1s
http_req_receiving	6ms	1s	0ms	0ms	9ms	31ms	91ms
http_req_sending	116µs	100ms	0ms	0ms	0ms	0ms	1ms
http_req_tls_handshaking	0ms	0ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	316ms	3s	259ms	0ms	640ms	813ms	1s
iteration_duration	1s	5s	1s	1s	2s	2s	3s

Counters

metric	count	rate
data_received	888 MB	8.79 MB/s
data_sent	2.66 MB	26.3 kB/s
http_reqs	9.7k	95.95/s
iterations	4.8k	47.98/s

Rates

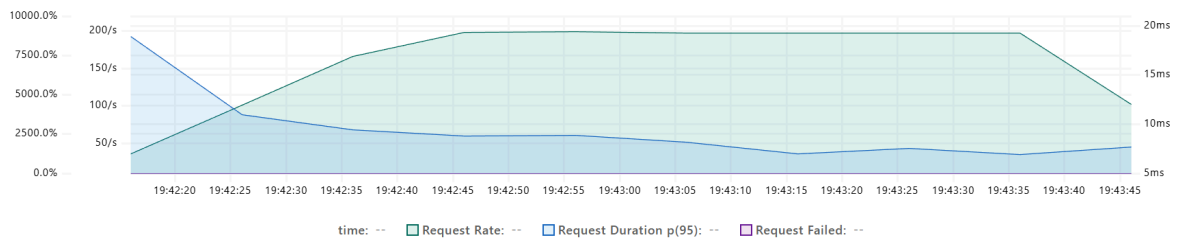
metric	rate
checks	100.0%
http_req_failed	0.0%

Gauges

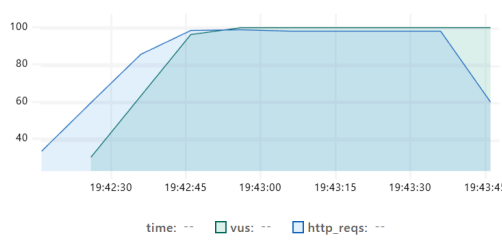
metric	value
vus	4
vus_max	100

Gráficos da 2ª Medição:

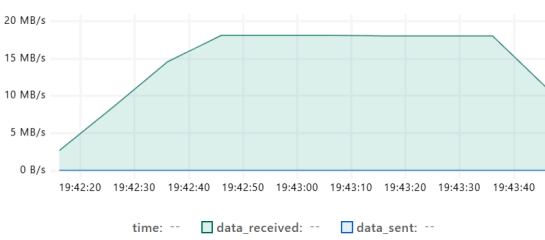
HTTP Performance overview



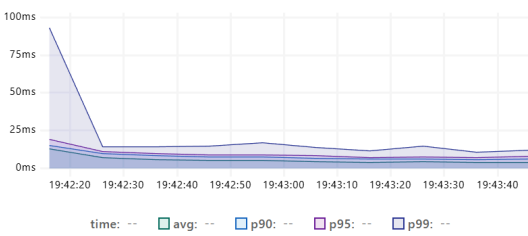
VUs



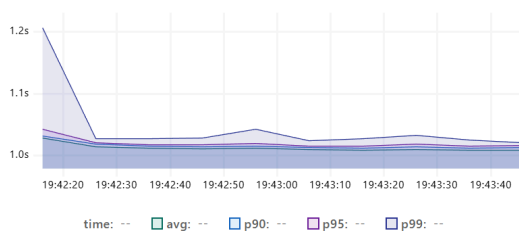
Transfer Rate



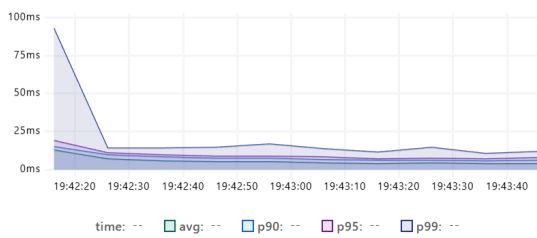
HTTP Request Duration



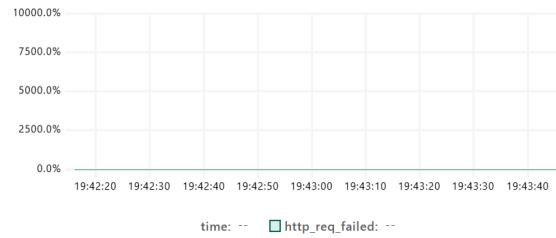
Iteration Duration



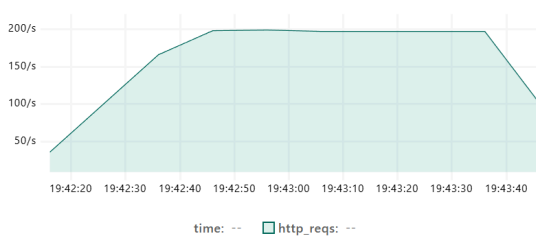
Request Duration



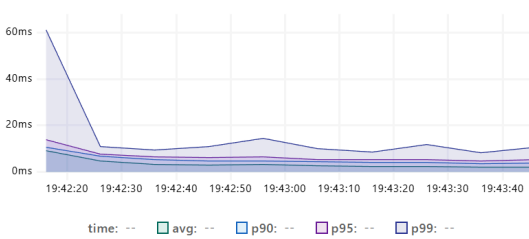
Request Failed Rate



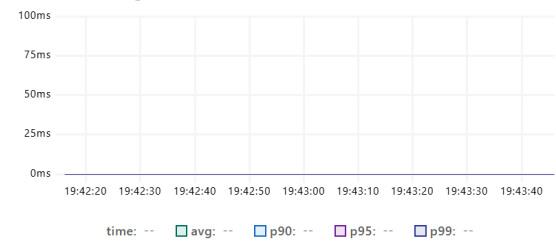
Request Rate



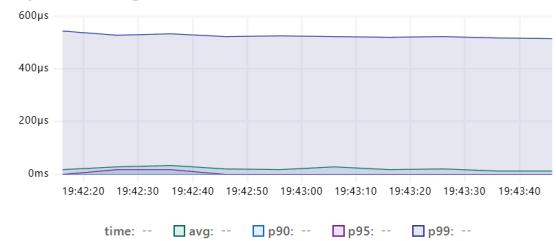
Request Waiting



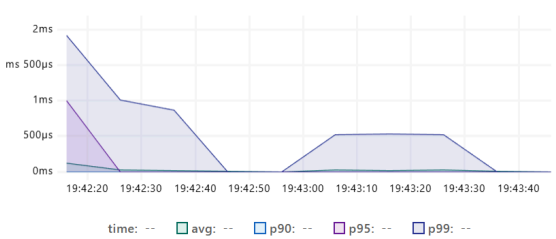
TLS handshaking



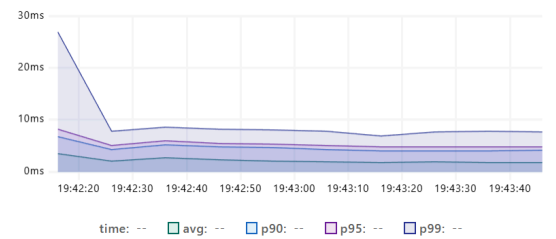
Request Sending



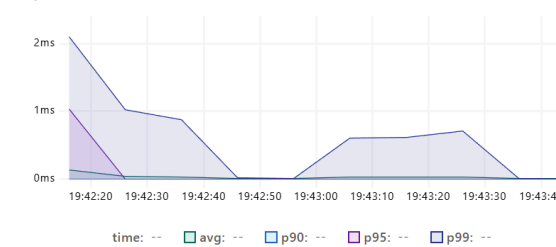
Request Connecting



Request Receiving



Request Blocked



Trends

metric	avg	max	med	min	p90	p95	p99
http_req_blocked	19μs	18ms	0ms	0ms	0ms	0ms	553μs
http_req_connecting	14μs	18ms	0ms	0ms	0ms	0ms	523μs
http_req_duration	4ms	456ms	4ms	0ms	7ms	9ms	15ms
http_req_receiving	2ms	56ms	1ms	0ms	4ms	5ms	8ms
http_req_sending	20μs	10ms	0ms	0ms	0ms	0ms	524μs
http_req_tls_handshaking	0ms	0ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	2ms	454ms	2ms	0ms	5ms	6ms	11ms
iteration_duration	1s	1s	1s	1s	1s	1s	1s

Counters

metric	count	rate
data_received	1.46 GB	14.5 MB/s
data_sent	4.37 MB	43.4 kB/s
http_reqs	15.9k	158.09/s
iterations	8k	79.05/s

Rates

metric	rate
checks	100.0%
http_req_failed	0.0%

Gauges

metric	value
vus	5
vus_max	100

Nome do Serviço 2: Listagem de Vulnerabilidades

- **Tipo de operações:** Leitura (GET).
- **Arquivos envolvidos:**
 - `src/main/java/com/example/backend_vuln/controller/VulnerabilidadeController.java`
 - `src/main/java/com/example/backend_vuln/repository/VulnerabilidadeRepository.java`
- **Arquivos com o código fonte de medição do SLA:**
 - `test_leitura.js` (Script k6)
- **Descrição das configurações:**
 - **Ambiente:** Execução Local (Windows).
 - **Runtime:** Java 21 (Spring Boot 3).
 - **Banco de Dados:** H2 Database (In-Memory).

Medição 1:

Testes de carga (SLA):

- **Latência (Tempo de resposta médio):** 236.92 ms
- **Pior caso (p95):** 822.36 ms
- **Vazão (Throughput):** 40 reqs/s
- **Concorrência:** 100 VUs (Usuários Virtuais)

Potenciais gargalos do sistema:

1. Largura de Banda e Serialização (Network/CPU Overhead) Embora a consulta ao banco em memória seja rápida, transformar uma lista grande de objetos Java em JSON e trafegar esses dados pela rede consome recursos. Se o banco de dados crescer (ex: 10.000 vulnerabilidades cadastradas), o *payload* de resposta ficará grande. Com 100 usuários baixando esse volume simultaneamente, o gargalo passará a ser a **largura de banda da rede** ou o tempo de CPU gasto apenas para montar o JSON gigante.

Medição 2:

Data da medição: 30/11/2025

Testes de carga (SLA):

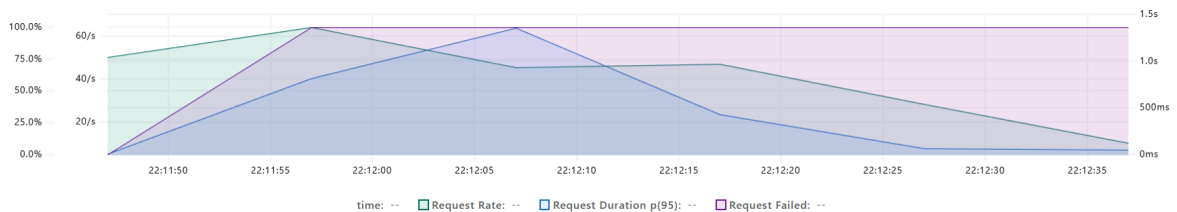
- **Latência (Tempo de resposta médio):** 9.79 ms
- **Pior Caso (p95):** 26.47 ms
- **Vazão (Throughput):** 50 reqs/s
- **Concorrência:** 100 VUs (Usuários Virtuais simultâneos)

Melhorias/otimizações:

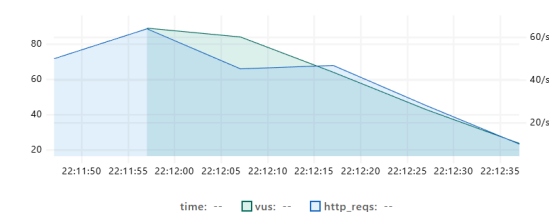
- src/main/java/com/example/backend_vuln/controller/VulnerabilidadeController.java
 - Alteração:** Refatoração do endpoint GET /vulnerabilidades para suportar paginação.
 - Otimização:** Substituição do método findAll() (que retornava a lista completa) pelo uso da interface Pageable. Isso limitou o payload de resposta a apenas 20 itens por página, reduzindo drasticamente o consumo de banda de rede e o tempo de serialização JSON.
- src/main/resources/application.properties
 - Alteração:** Manutenção dos parâmetros de tuning de performance (já aplicados na fase anterior).
 - Otimização:**
 - Pool de conexões (HikariCP) ajustado para 20.
 - Threads do Tomcat ajustadas para 200.
- src/main/java/com/example/backend_vuln/repository/VulnerabilidadeRepository.java
 - Alteração:** Uso implícito da interface PagingAndSortingRepository (através do JpaRepository).
 - Otimização:** Permitiu que o banco de dados (H2) executasse queries otimizadas com LIMIT e OFFSET, em vez de carregar a tabela inteira na memória RAM da aplicação.

Gráficos da 1ª Medição:

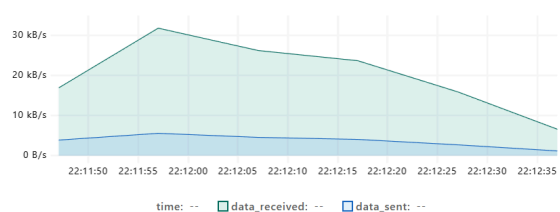
HTTP Performance overview



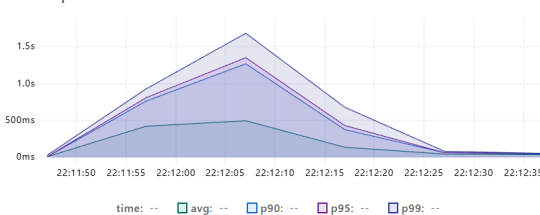
VUs



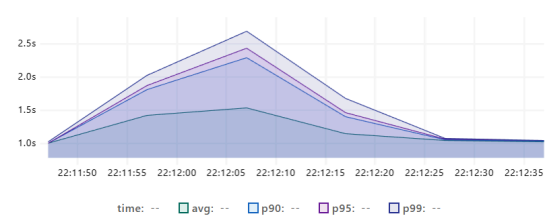
Transfer Rate



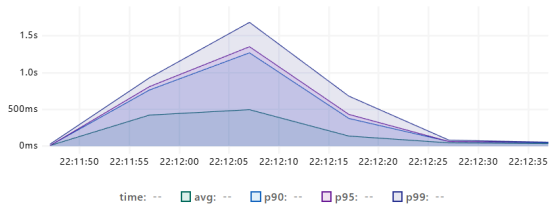
HTTP Request Duration



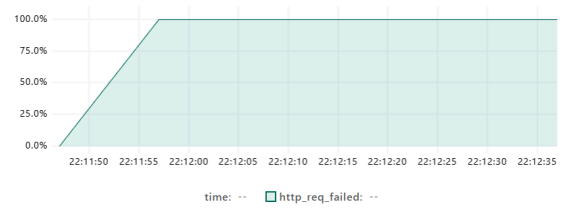
Iteration Duration



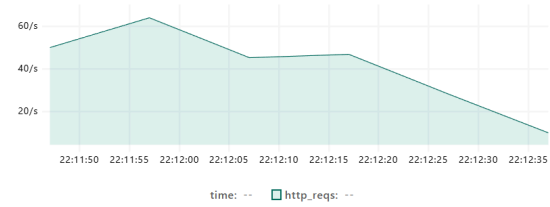
Request Duration



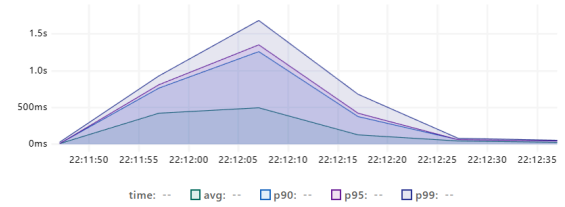
Request Failed Rate



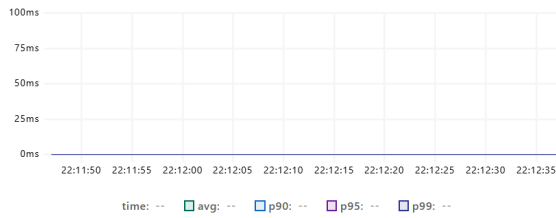
Request Rate



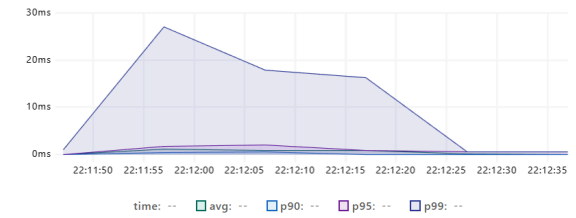
Request Waiting



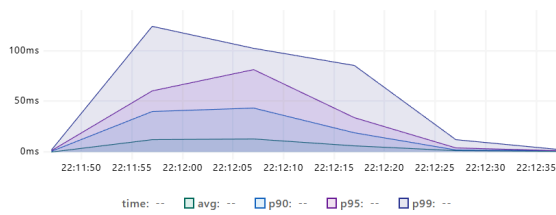
TLS handshaking



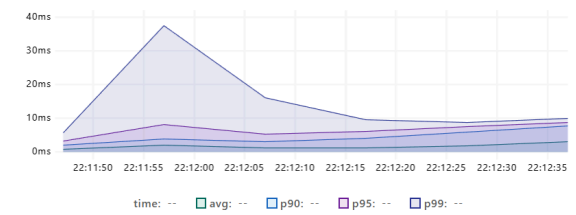
Request Sending

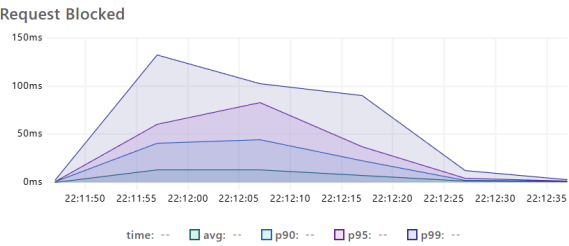


Request Connecting



Request Receiving



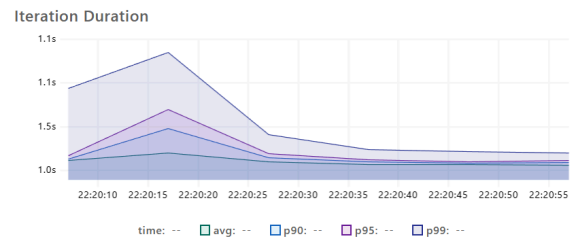
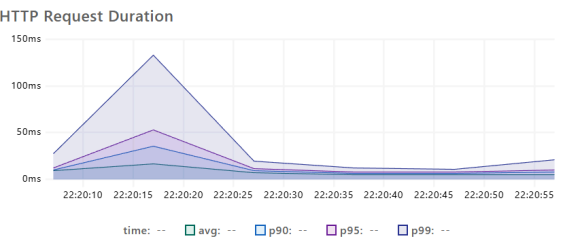
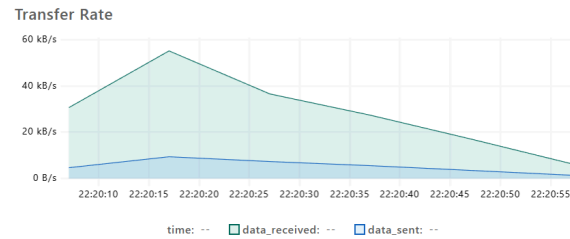
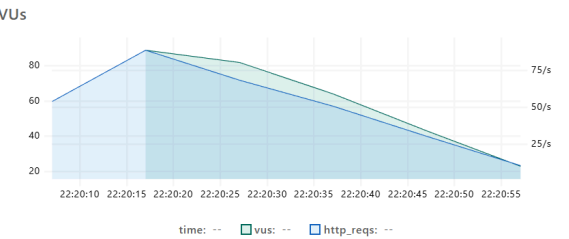
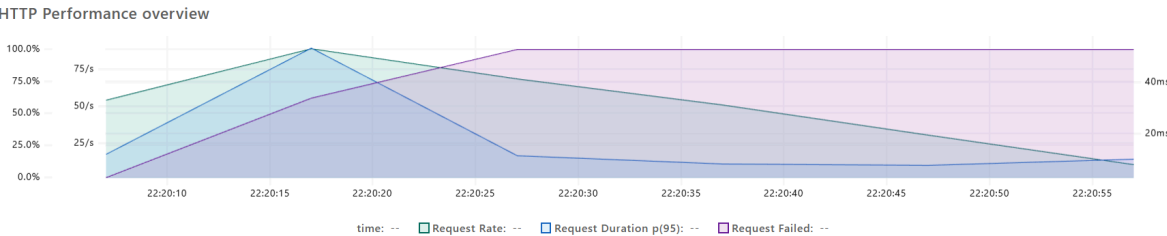


Trends							
metric	avg	max	med	min	p90	p95	p99
http_req_blocked	7ms	329ms	788µs	0ms	20ms	45ms	94ms
http_req_connecting	6ms	329ms	747µs	0ms	18ms	40ms	85ms
http_req_duration	236ms	1s	69ms	0ms	684ms	822ms	1s
http_req_receiving	1ms	101ms	0ms	0ms	4ms	6ms	12ms
http_req_sending	643µs	160ms	0ms	0ms	0ms	546µs	15ms
http_req_tls_handshaking	0ms	0ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	234ms	1s	68ms	0ms	683ms	822ms	1s
iteration_duration	1s	2s	1s	1s	1s	1s	2s

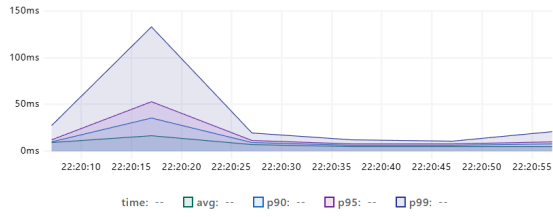
Counters		
metric	count	rate
data_received	1.22 MB	20 kB/s
data_sent	221 kB	3.63 kB/s
http_reqs	2.5k	40.35/s
iterations	2.5k	40.35/s

Rates		Gauges	
metric	rate	metric	value
checks	20.4%	vus	1
http_req_failed	79.6%	vus_max	100

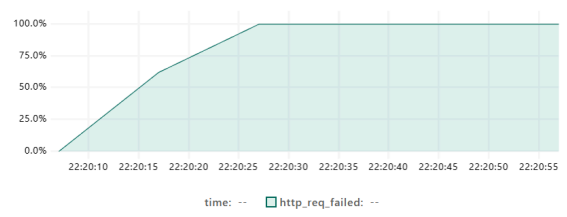
Gráficos da 2ª Medição:



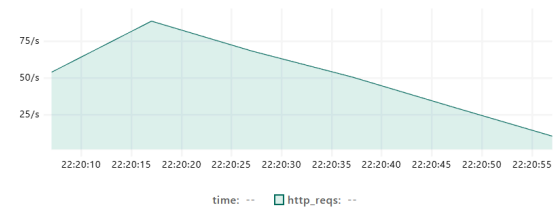
Request Duration



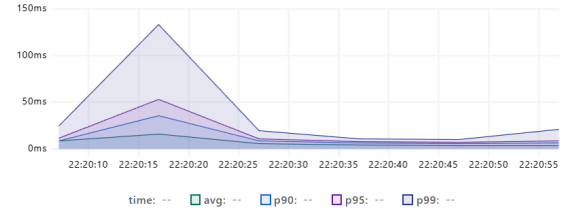
Request Failed Rate



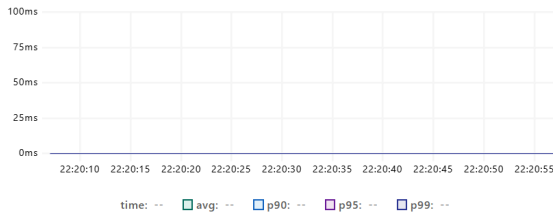
Request Rate



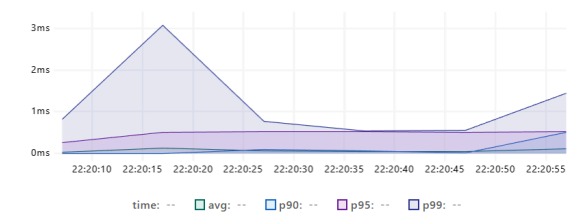
Request Waiting



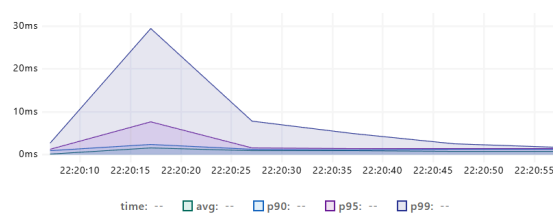
TLS handshaking



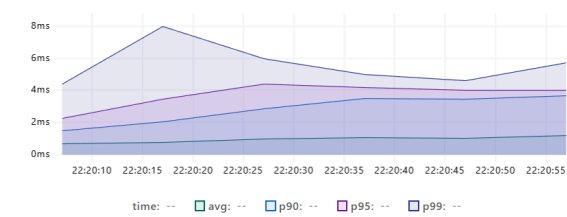
Request Sending



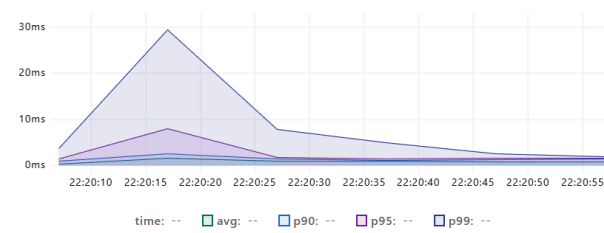
Request Connecting



Request Receiving



Request Blocked



Trends

metric	avg	max	med	min	p90	p95	p99
http_req_blocked	1ms	53ms	542µs	0ms	1ms	2ms	17ms
http_req_connecting	995µs	53ms	534µs	0ms	1ms	2ms	16ms
http_req_duration	9ms	537ms	5ms	0ms	15ms	26ms	76ms
http_req_receiving	903µs	24ms	275µs	0ms	2ms	3ms	5ms
http_req_sending	77µs	23ms	0ms	0ms	11µs	519µs	1ms
http_req_tls_handshaking	0ms	0ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	8ms	533ms	5ms	0ms	14ms	24ms	76ms
iteration_duration	1s	1s	1s	1s	1s	1s	1s

Counters

metric	count	rate
data_received	1.74 MB	28.9 kB/s
data_sent	318 kB	5.28 kB/s
http_reqs	3k	50.33/s
iterations	3k	50.33/s

Rates

metric	rate
checks	28.9%
http_req_failed	71.1%

Gauges

metric	value
vus	2
vus_max	100